

# EdvantiQ — Complete System Design Specification (v1)

## 1. Overview

EdvantiQ is a **multi-tenant School Management System (SaaS)** designed to serve educational institutions of all levels while maintaining strict data isolation, role-based access control, and a highly configurable academic structure.

It is built as a centralized platform where each school operates as an independent tenant with full autonomy over its data and operations.

---

## 2. Tech Stack

### Frontend

- React (UI library)
- Vite (build tool)
- Tailwind CSS (styling)

### Backend

- Node.js (runtime)
- Express.js (web framework)

### Database

- PostgreSQL (relational database)

### ORM

- Prisma (recommended type-safe ORM)

### Authentication & Security

- JWT (access + refresh tokens)
- bcrypt (password hashing)
- Role-Based Access Control (RBAC)

### Supporting Tools

- dotenv (environment variables)
- CORS (cross-origin handling)
- Cloud storage (AWS S3 / Cloudinary - future)

- Deployment: Vercel (frontend), Render/Railway (backend)
- 

### 3. System Architecture

#### Core Design

- Multi-tenant SaaS architecture
- Each school is a tenant
- All data is isolated using `school_id`

#### Layers

1. Platform Layer (Super Admin)
  2. Tenant Layer (School)
  3. Role Layer (Users inside school)
  4. Permission Layer (Granular access control)
- 

### 4. Multi-Tenant Model

#### Rule

Every table contains:

- `school_id`

#### Isolation Guarantee

- No cross-school data access
  - Backend enforces tenant filtering on every query
- 

### 5. Governance Model

#### 5.1 Super Admin (Platform Owner)

- Exists only at platform level
- Creates schools
- Views system-wide analytics
- Has NO default access inside schools
- Must request permission from School Admin to access data
- Access is temporary and fully audited

## **Access Flow**

- Super Admin requests access
  - School Admin approves/denies
  - If approved:
    - time-limited session granted
    - all actions logged
- 

## **5.2 School Admin**

- Full control within school
  - Manages students, teachers, staff, academics, fees, communication
  - Creates users and assigns roles
- 

## **5.3 Teachers**

- Mark attendance
  - Enter grades
  - Manage class academic activities
- 

## **5.4 Students**

- View results
  - View attendance
  - View fees
- 

## **5.5 Parents**

- View child performance
  - View attendance
  - View fees
- 

## **5.6 Staff**

- Role-based access (accountant, librarian, etc.)
- 

## **6. School Types**

- Basic School (Primary + JHS)
- Secondary School (SHS)

- Tertiary Institution (University/College)

### **Rule**

- School type defines default structure
  - Admin can fully customize internal structure
- 

## **7. School Setup Flow**

### **Step 1**

Super Admin creates school:

- Name
- Type
- Location
- Admin account

### **Step 2**

Admin login

- Must change password on first login

### **Step 3**

Setup Wizard

- Academic structure
- Classes
- Subjects
- Users
- System preferences

System remains usable before full completion

---

## **8. Academic System**

### **Structure**

- Fully dynamic per school
- Supports:
  - Continuous Assessment (CA)
  - Exams
  - Quizzes, assignments, tests

## Grading

- Custom grading per school
  - Dynamic CA breakdown (e.g. Quiz 40%, Exam 60%)
- 

## 9. Attendance System

### Basic Schools

- Daily class attendance
- Marked by class teacher

### SHS/University

- Per lesson/lecture attendance

### Rule

- Mode configurable by School Admin
- 

## 10. Fees System

- Fee structures
  - Payments tracking
  - Installments
  - Receipts
  - Balances
  - Optional scholarships/discounts
- 

## 11. Communication System

- Announcements
  - Targeted messaging (class, individual, school-wide)
  - Result release notifications
- 

## 12. Dashboards

### School Admin

- Students overview
- Teachers overview
- Attendance stats

- Fees overview
- Academic performance

### **Teacher**

- Assigned classes
- Attendance tasks
- Grading tasks

### **Student**

- Results
- Attendance
- Fees

### **Parent**

- Child performance
- Attendance
- Fees

---

## **13. Permission System (RBAC)**

### **Concept**

Roles = permissions group Permissions = atomic actions

### **Examples**

- student:create
- attendance:mark
- grades:update
- fees:record\_payment

### **Rules**

- School Admin can customize roles
- Every request checked via permissions + school\_id

---

## **14. Authentication System**

- Single login entry
- No signup
- Admin creates users
- First login forces password change

- JWT authentication
- 

## 15. Data Rules

- Soft delete only ( `is_deleted` )
  - Recoverable data
  - Audit logs recommended
- 

## 16. Backend Architecture

### Modules

- auth
- schools
- users
- students
- teachers
- staff
- classes
- subjects
- attendance
- academics
- fees
- communication

### Middleware

- authMiddleware
  - tenantMiddleware
  - permissionMiddleware
- 

## 17. Database Schema (Core Tables)

- users
- schools
- school\_admins
- students
- teachers
- staff
- classes
- subjects
- attendance
- assessments

- marks
- fees
- payments
- announcements

All tables include:

- school\_id
- 

## 18. Data Flow Examples

### Exams Flow

Teacher → marks entry → assessment engine → grading → report cards → notifications

### Attendance Flow

Teacher → marks attendance → stored → reports generated

---

## 19. Key Principles

- Multi-tenant isolation
  - Zero-trust Super Admin model
  - Fully dynamic academic configuration
  - Permission-based security
  - Soft delete everywhere
  - Modular backend design
  - Scalable SaaS architecture
- 

## 20. Final Summary

EdvantiQ is a full SaaS educational ERP system that:

- Supports multiple independent schools
  - Ensures strict data isolation
  - Uses role + permission-based access control
  - Provides full academic + financial management
  - Allows Super Admin oversight via controlled access requests
-

## Status

This document represents the **complete system design specification (v1) for EdvantiQ.**